

Snowflake Lab: Zero-Copy Cloning, Streams & Tasks, and SQL UDF

Task 1: Create a Clone of an Existing Database Using Zero-Copy Cloning

Step 1: Create the Original Database and Table

SQL Command

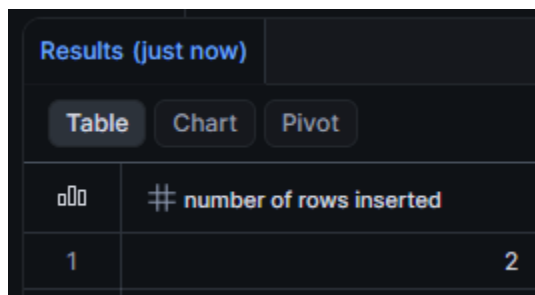
```
CREATE OR REPLACE DATABASE original_database;
```

```
USE DATABASE original_database;
```

```
CREATE OR REPLACE SCHEMA demo_schema;
```

```
CREATE OR REPLACE TABLE original_table (  
  id INTEGER,  
  product_name STRING,  
  column_name STRING  
);
```

```
INSERT INTO original_table VALUES  
(1, 'Laptop', 'Old Value'),  
(2, 'Phone', 'Old Value');
```



The screenshot shows the Snowflake query results interface. At the top, it says "Results (just now)". Below that are three tabs: "Table", "Chart", and "Pivot". The "Table" tab is selected. The table has two columns: the first column is labeled with a square icon and the number "1", and the second column is labeled with a hash icon and the text "number of rows inserted". The table contains two rows of data.

1	# number of rows inserted
1	2

Step 2: Create a Database Clone

SQL Command

```
CREATE OR REPLACE DATABASE cloned_database  
CLONE original_database;
```

Output

	 status
1	Database CLONED_DATABASE successfully created.

Step 3: Modify Data in the Clone

SQL Command

```
USE DATABASE cloned_database;  
USE SCHEMA demo_schema;
```

```
UPDATE original_table  
SET column_name = 'New Value'  
WHERE id = 1;
```

Output

Results (just now)		
Table Chart Pivot		
	# number of rows updated	# number of multi-joined rows updated
1	1	0

Step 4: Verify Data in the Clone

SQL Command

```
SELECT * FROM original_table;
```

Result

Results (just now)			
Table Chart Pivot			
	# ID	PRODUCT_NAME	COLUMN_NAME
1	1	Laptop	New Value
2	2	Phone	Old Value

Step 5: Verify Original Data Remains Unchanged

SQL Command

```
USE DATABASE original_database;
USE SCHEMA demo_schema;
```

```
SELECT * FROM original_table;
```

Result

Table Chart Pivot			
	# ID	PRODUCT_NAME	COLUMN_NAME
1	1	Laptop	Old Value
2	2	Phone	Old Value

Observation

Changes made to the cloned database did not affect the original database. This confirms the successful implementation of Zero-Copy Cloning.

Task 2: Set Up a Stream and Task for Continuous Integration

Step 1: Create Source and Target Tables

SQL Command

```
CREATE OR REPLACE TABLE my_table (  
  id INTEGER,  
  name STRING  
);
```

```
CREATE OR REPLACE TABLE target_table (  
  id INTEGER,  
  name STRING  
);
```

Output

Table Chart Pivot	
	 status
1	Table TARGET_TABLE successfully created.

Step 2: Create a Stream

SQL Command

```
CREATE OR REPLACE STREAM my_stream  
ON TABLE my_table;
```

Output

Results (just now)	
Table Chart Pivot	
	 status
1	Stream MY_STREAM successfully created.

Step 3: Insert Sample Data

SQL Command

```
INSERT INTO my_table VALUES  
(1, 'John'),  
(2, 'Alice');
```

Step 4: Create a Task

SQL Command

```
CREATE OR REPLACE TASK my_task  
WAREHOUSE = my_warehouse  
SCHEDULE = '5 MINUTE'  
AS  
INSERT INTO target_table  
SELECT id, name  
FROM my_stream  
WHERE METADATA$ACTION = 'INSERT';
```

Output

Results (just now)

Table

Chart

Pivot

	<u>A</u> status
1	Task MY_TASK successfully created.

Step 5: Enable the Task

SQL Command

```
ALTER TASK my_task RESUME;
```

Output

Task activated and scheduled to run every 5 minutes.

Step 6: Monitor the Stream

SQL Command

SELECT * FROM my_stream;

Output

Restart (just now)						
Table Chart Pivot						
id	#	id	NAME	METADATASACTION	METADATASISUPDATE	METADATASROW_ID
1		1	John	INSERT	FALSE	4e06a9d239bba2f1ee38da3f1b0cec627beb3c5f
2		2	Alice	INSERT	FALSE	31bf4a82bf3eb85da7c6d811033f3beb6403c016

Step 7: Monitor Task Status

SQL Command

SHOW TASKS;

Output

id	created_on	name	id	database_name	schema_name	owner	comment	warehouse	schedule	predecessors	state	definition
1	2026-08-05 01:37:17	MY_TASK	01c4d7a5-b90f-b297	ORIGINAL_DATABASE	DEMO_SCHEMA	ACCOUNTADMIN		MY_WAREHOUSE	5 MINUTE		started	INSERT INTO target_ta

Task 3: Implement a Simple SQL UDF

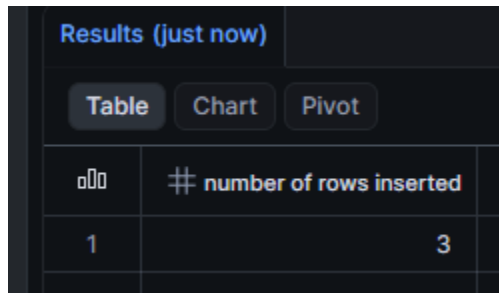
Step 1: Create Products Table

SQL Command

```
CREATE OR REPLACE TABLE products (  
  product_name STRING,  
  price DECIMAL(10,2)  
);
```

```
INSERT INTO products VALUES  
( 'Laptop', 1000.00),  
( 'Phone', 500.00),  
( 'Tablet', 300.00);
```

Output



The screenshot shows a database interface with a table titled "Results (just now)". The table has one column labeled "# number of rows inserted" and one row with the value "3".

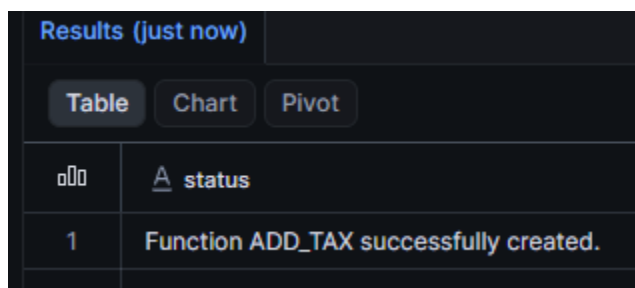
number of rows inserted
3

Step 2: Create a SQL UDF

SQL Command

```
CREATE OR REPLACE FUNCTION add_tax(price DECIMAL)
RETURNS DECIMAL
AS
$$
    SELECT price * 1.1
$$;
```

Output



The screenshot shows a database interface with a table titled "Results (just now)". The table has one column labeled "status" and one row with the value "Function ADD_TAX successfully created.".

status
Function ADD_TAX successfully created.

Step 3: Use the UDF

SQL Command

```
SELECT
    product_name,
    price,
    add_tax(price) AS price_with_tax
FROM products;
```

Result

Results (just now)			
Table Chart Pivot			
❏	PRODUCT_NAME	PRICE	PRICE_WITH_TAX
1	Laptop	1000.00	1100.000
2	Phone	500.00	550.000
3	Tablet	300.00	330.000

Observation

The UDF successfully calculated product prices with a 10% tax applied.

Conclusion

This lab successfully demonstrated:

- Creation of database clones using Snowflake Zero-Copy Cloning.
- Independent modification of cloned data without affecting the source database.
- Implementation of Streams for change data capture.
- Automation of data movement using Tasks.
- Creation and usage of a SQL User Defined Function (UDF).

All objectives were completed successfully.